

GUÍA 02 · DECISIONES TÉCNICAS

Tecnologías y decisiones

Cómo funciona cada elección y cómo justificarla frente a alternativas

Proyecto TFG · versión alineada con el código actual

Alejandro Villarrubia García

Universidad Europea Miguel de Cervantes

Índice

1. Criterio de selección	2
2. Comparativa de tecnologías	2
3. Por qué dos modelos lingüísticos	3
4. SOLID aplicado	3
5. Decisiones de seguridad	3
6. Reproducibilidad y evaluación	4
7. Diferencias con la propuesta inicial	4
8. Qué no prometer	5

1. Criterio de selección

La arquitectura prioriza reproducibilidad local, facilidad de defensa, separación de responsabilidades y coste operativo bajo. Las decisiones no intentan convertir el TFG en un servicio de producción multiusuario: delimitan un detector demostrable, auditable y extensible.

Frente a la propuesta aprobada, el cumplimiento funcional es parcial: quedaron fuera la reputación online, la validación TLS y la inspección activa de sitios. Se excluyeron para mantener el análisis estático y evitar dependencias externas, con menor cobertura como consecuencia. La memoria lo documenta en 5.3.1; el apartado 5.3.2 justifica TensorFlow y 5.10 reúne los cinco modelos UML del diseño final.

2. Comparativa de tecnologías

Cuadro 1: Comparativa de tecnologías

Tecnología	Cómo se usa	Por qué se elige	Alternativa y límite
Python	Integra parser, reglas, ML, API y automatizaciones	Ecosistema científico, lectura clara y buena trazabilidad	Java/Node serían válidos; añadirían complejidad para este alcance
Streamlit	Cliente web con sistema visual compartido y adaptable	Permite validar el flujo cliente-servidor con una interfaz coherente y poco código	React daría comunicación navegador-API directa, pero exigiría otro stack
scikit-learn	TF-IDF vectoriza y MLPClassifier clasifica	Pipeline reproducible, API madura y métricas estándar	TensorFlow permite otras arquitecturas; no se implementó ni midió una alternativa equivalente.
TF-IDF	Convierte palabras y bigramas en variables	Explicable, rápido y adecuado para corpus moderados	Transformers ofrecen representaciones contextuales; su posible mejora requeriría una comparación independiente.
email estándar	Parsea MIME y cabeceras EML	No añade dependencia para la parte esencial del formato	Librerías externas pueden aportar atajos, pero no son necesarias
BeautifulSoup	Inspecciona HTML y formularios	Tolera HTML imperfecto de correos	Parser regex puro sería frágil; un navegador sería inseguro e innecesario
Gmail API + OAuth	Obtiene mensajes con consentimiento del usuario	Permisos oficiales y flujo revocable	IMAP sería más genérico, pero no aporta el mismo control API
HTTP stdlib	Expone análisis y administración central	Cero framework extra, contrato pequeño y fácil de auditar	FastAPI facilitaría OpenAPI y escalado; no es necesario para el prototipo
joblib	Persiste pipeline TF-IDF + MLP	Guarda vectorizador y modelo como una unidad	ONNX sería más portable; exige conversión y no elimina el requisito de confiar en artefactos

Tecnología	Cómo se usa	Por qué se elige	Alternativa y límite
unittest + Ruff	Prueba comportamiento y calidad estática	unittest forma parte de Python; Ruff es una dependencia de desarrollo instalada por separado.	pytest aporta fixtures más ricas; el conjunto actual es suficiente y directo

3. Por qué dos modelos lingüísticos

El detector identifica idioma por mensaje, no por sesión. Mantiene un modelo español y otro inglés porque las palabras, stopwords y corpus tienen distribuciones distintas. EmailAnalysisService cachea un detector por idioma para combinar corrección con rendimiento.

Nota para la defensa

Respuesta breve: La alternativa de un único modelo multilingüe simplificaría ficheros, pero mezclaría vocabularios y haría más difícil justificar qué datos explican cada predicción.

4. SOLID aplicado

Cuadro 2: Aplicación de SOLID

Principio	Aplicación en el código	Beneficio demostrable
Responsabilidad única	Parser, señales, scorer, explicaciones, ML, monitor y transporte están separados	Un cambio en URLs no obliga a tocar Gmail o Streamlit
Abierto/cerrado	EmailAnalysisService recibe analyzer, detector loader y detector de idioma inyectables	Se prueban estrategias y se añaden modelos sin reescribir el coordinador
Sustitución	Los consumidores dependen de BackendClient o su protocolo mínimo	Web, extensión y monitor consumen el mismo contrato HTTP
Segregación	El almacenamiento, el análisis y el transporte tienen superficies pequeñas	Cada prueba necesita solo la interfaz que utiliza
Inversión	NeuralModelTrainer depende de ModelStorage, no de una ruta fija	Facilita memoria, disco y dobles de prueba

5. Decisiones de seguridad

- El parser limita EML a 10 MiB; /analyze admite 16 MiB y las rutas de datasets 256 MiB con Content-Length obligatorio.
- La API no acepta CORS *; permite Gmail y orígenes chrome-extension://, y no devuelve trazas internas.
- Por defecto Streamlit y el backend escuchan en loopback. Para una demo móvil puede abrirse solo Streamlit en 0.0.0.0:8501 y mantener el backend en 127.0.0.1:8766; no requiere --allow-remote, no es Internet y debe limitarse a una LAN privada de confianza. Un backend remoto sí exige HTTPS y controles adicionales.
- El modelo no ejecuta archivos adjuntos; solo registra metadatos y señales.

- Los nuevos joblib no conservan textos brutos de entrenamiento. Los modelos antiguos se sanean al cargarse y guardarse con el formato actual.
- Credenciales, tokens, estados y artefactos temporales de QA se mantienen fuera del índice Git mediante .gitignore.

6. Reproducibilidad y evaluación

Los hiperparámetros finales son TF-IDF (ngram_range 1-2, max_features 3.000, min_df 1, strip_accents unicode) y MLP ((64, 32), relu, alpha 0,0001, learning rate 0,001, 500 iteraciones, sin early stopping, semilla 42). El protocolo usa 1.148/209 textos ES y 65.661/16.416 EN. La clase positiva mezcla spam/phishing según la fuente y se documenta como proxy, no como equivalencia. Los hashes permiten identificar los joblib y los modelos no guardan textos brutos.

El benchmark reproducible separa arranque e inferencia. Se midió en Windows 11 Home 25H2 de 64 bits (26200.9278), Ryzen 7 7800X3D, 63,1 GiB RAM y Python 3.12.7 con scikit-learn 1.9.0, Streamlit 1.58.0, NumPy 2.4.6, SciPy 1.17.1 y joblib 1.5.3. La carga diferida redujo importación heurística un 96,6 % y arranque de aplicación un 76,2 %; la inferencia varió hasta aproximadamente un 3 %.

La calibración usa 40 casos separados en cinco particiones: pesos heurísticos 20-50, umbrales 20-60 y alta confianza 65-85. Selecciona 45/55, umbral 21 y alta confianza 70 con desempates predefinidos; no produce una probabilidad calibrada. La prueba de concurrencia usa el backend real: 8 y 16 análisis terminaron sin errores, pero a 32 aparecieron fallos. Se recomiendan 4-8 para demo.

La selección 45/55 es local y debe repetirse si cambian datos, reglas o modelos. En los 16 EML reservados, el heurístico obtuvo 100 % de exactitud y el combinado 87,5 %, con dos falsas alarmas. No se demuestra una mejora universal de la combinación.

La validación automatizada reúne 94 pruebas Python y 2 pruebas con Chromium. Incluye idioma determinista y el recorrido local Gmail EML -> JSON -> HTTP -> backend -> Telegram. GitHub Actions instala versiones fijadas, ejecuta Ruff, verifica calibración, regenera la evaluación, comprueba el respaldo de defensa y recorre web-backend. OAuth real usa una lista E2E separada porque sus credenciales no pueden entrar en CI.

7. Diferencias con la propuesta inicial

Cuadro 3: Diferencias frente a la propuesta inicial

Propuesta	Implementación actual	Cómo explicarlo
TensorFlow	scikit-learn MLP	TF-IDF y MLP comparten un pipeline local. No se midió superioridad frente a TensorFlow.
Blacklists/reputación	Señales locales de dominio, URL, punycode y acortadores	Se evita depender de red externa y se mantiene privacidad; reputación online queda como extensión
Certificados	No se inspecciona TLS desde el correo	El EML no prueba por sí mismo el certificado del destino; se prioriza análisis estático seguro
Interfaz simple	Streamlit como cliente de una API central obligatoria	La interfaz sigue siendo simple y todos los canales comparten los modelos del servidor

8. Qué no prometer

- El prototipo no implementa consultas online de reputación; añadir credenciales no activa esa capacidad.
- No presentar el accuracy de entrenamiento como precisión en producción.
- No llamar a la extensión un producto publicado: se carga en modo desarrollador y consume el backend central configurado.